Faculdade de Tecnologia de Carapicuíba - FATEC

Curso de Tecnologia em Jogos Digitais

Processamento Paralelo em Jogos Digitais
Prof. Dr. Jean M. Laine

Trabalho Prático de Lab (TP)
Processamento Paralelo de Logs de Loot

1. Introdução e Contexto: O Gargalo de Amdahl

Cenário: A Crise do Pós-Battle

Após a queda do lendário dragão *Amdahl, o Devorador de Núcleos*, o servidor central do seu MMORPG foi inundado por registros de transações (*loot*).
O Problema: O sistema sequencial atual funciona como um único tesoureiro contando milhões de moedas sozinho. O resultado? Um atraso de processamento que congela a economia do reino.
Sua Missão: Contratar um "exército de tesoureiros"(*Multithreading*) para processar esses dados simultaneamente. O desafio é velocidade com **integridade**: se os dados forem somados incorretamente (Race Condition), a economia colapsa.

2. Especificação do Formato do Arquivo de Log

O arquivo de log segue o formato CSV com separador ponto-e-vírgula (;). Cada linha representa um evento de *loot* obtido por um jogador.

Estrutura da Linha

```
ID_Item;Valor;Jogador;Timestamp
```

- **ID_Item:** Identificador único do item (string sem espaços). Ex: `Espada_Flamejante`
- **Valor:** Valor numérico do item em moedas de ouro (double com 2 casas decimais). Ex: `150.50`
- **Jogador:** Nome do personagem que obteve o item (string sem espaços). Ex: `Thorin_Escudo`
- **Timestamp:** Data e hora do evento no formato `YYYY-MM-DD HH:MM:SS`. Ex: `2024-01-15 14:30:22`

Atenção: Validação de Dados

- Linhas malformadas devem ser ignoradas silenciosamente
- Apenas a coluna **Valor** é relevante para o cálculo do imposto
- O imposto calculado é de **5%** sobre cada valor (taxa da guilda)

2.1 Exemplo de Conteúdo do Arquivo

```
Espada_Flamejante;1250.00;Thorin_Escudo;2024-01-15 14:30:22
Pocao_de_Vida;50.00;Legolas_Verde;2024-01-15 14:31:05
Moeda_de_Ouro;1.00;Gimli_Martelo;2024-01-15 14:32:18
Gema_Rara;5000.00;Gandalf_Branco;2024-01-15 14:35:42
```

3. Justificativa do Paralelismo e Lei de Amdahl

O paralelismo permite que cada núcleo da CPU processe uma fatia independente dos dados simultaneamente. Em um cenário de 10 milhões de itens, isso reduz drasticamente o tempo total de resposta.

3.1 Fundamentação Teórica - Lei de Amdahl

A aceleração máxima teórica (*speedup*) é limitada pela fração sequencial do código:

$$S(n) = \frac{1}{(1-P) + \frac{P}{n}}$$

Onde:

- $S(n)$ = Speedup com n processadores
- P = Fração paralelizável do código (0 a 1)
- $1 - P$ = Fração sequencial

Exemplo prático: Se 95% do código é paralelizável ($P = 0.95$), o speedup máximo teórico com 8 threads é:

$$S(8) = \frac{1}{(1-0.95) + \frac{0.95}{8}} = \frac{1}{0.05 + 0.11875} \approx 5.93$$

Ou seja, mesmo com 8 núcleos, o ganho máximo é de aproximadamente 6x devido à parte sequencial.

4. Requisitos Técnicos (Pontuação: 5.0)

4.1 Obrigatórios

- **Interface:** Receber via `args[]` o caminho do arquivo e o número de threads.
- **Implementação:** Uso obrigatório da classe `System.Threading.Thread` e cálculo manual de índices.
- **Sincronização:** Uso obrigatório de `lock` para proteger a região crítica.
- **Desafio do Resto:** O algoritmo deve garantir que todos os itens sejam processados, tratando o resto da divisão entre o total de itens e o número de threads.

4.2 Tratamento do Resto (Exemplo Didático)

```
int tamanhoFatia = totalItens / numThreads;
int resto = totalItens % numThreads;

for (int i = 0; i < numThreads; i++) {
    int inicio = i * tamanhoFatia;
    int fim = inicio + tamanhoFatia;

    // Distribui o resto entre as primeiras threads
    if (i < resto) {
        inicio += i;
        fim += i + 1;
    } else {
        inicio += resto;
        fim += resto;
    }

    // Dispara thread para processar [inicio, fim)
}
```

5. Saídas Padrão e Validação (Obrigatório)

Para garantir a integridade dos dados, todas as versões do programa (Sequencial e Paralela) devem obrigatoriamente apresentar as seguintes saídas no console:

- **Quantidade de Itens Válidos:** Total de linhas processadas (ignorando malformadas)
- **Total de Impostos Arrecadados:** A soma final dos impostos (5% de cada valor)
- **Tempo de Execução:** Medido apenas na etapa de cálculo matemático (excluindo leitura do disco)
- **Comparação de Resultados:** O total da versão paralela deve ser IDÊNTICO ao sequencial

5.1 Formato Obrigatório da Saída

```
=== SISTEMA SEQUENCIAL ===
Itens válidos: 9,999,999
Total de impostos: 2,534,167.89
Tempo de cálculo: 1250 ms

=== SISTEMA PARALELO (8 threads) ===
Itens válidos: 9,999,999
Total de impostos: 2,534,167.89
Tempo de cálculo: 187 ms
Speedup: 6.68x
```

6. Exemplos de Implementação

6.1 Carregamento de Dados

```
public static List<double> CarregarDados(string caminho) {
    var lista = new List<double>();
    // File.ReadLines nao carrega o arquivo inteiro na RAM
    foreach (var linha in File.ReadLines(caminho)) {
        var colunas = linha.Split(';');
        if (colunas.Length >= 2 &&
            double.TryParse(colunas[1], out double valor)) {
            lista.Add(valor);
        }
    }
    return lista;
}
```

6.2 Processamento com Lock (Versão Didática)

```
object cadeado = new object();
double totalGeral = 0;

// Cada thread executa esta logica
void ProcessarFatia(int inicio, int fim) {
    for (int i = inicio; i < fim; i++) {
        double imposto = valores[i] * 0.05;
        lock (cadeado) {
            totalGeral += imposto; // Regiao critica
        }
    }
}
```

Atenção: Performance do Lock

Observe que o `lock` dentro do loop cria uma contenção severa. **Desafio:** Como reorganizar o código para reduzir a contenção e melhorar o speedup?

6.3 Medição de Tempo

```
Stopwatch sw = Stopwatch.StartNew();
// ... Execucao das threads e t.Join() ...
sw.Stop();

Console.WriteLine($"Tempo: {sw.ElapsedMilliseconds} ms");
Console.WriteLine($"Speedup: {tempoSequencial / (double)tempoParalelo:
    F2}x");
```

7. Estrutura do Relatório (Pontuação: 5.0)

O relatório deve conter as seguintes seções:

1. **Introdução Técnica:** Explicação sobre Threads, Sincronização e Lei de Amdahl.
2. **Metodologia:**
   - Descrição da lógica de divisão de índices
   - Tratamento do resto da divisão
   - Estratégia de sincronização utilizada
3. **Ambiente de Teste:** Especificações da máquina (CPU, RAM, SO, versão do .NET)
4. **Tabela Comparativa:** Resultados para 1, 2, 4, 8 e 16 threads. Exemplo:

Threads	Tempo (ms)	Speedup	Eficiência
1 (sequencial)	1250	1.00	100%
2	630	1.98	99%
4	320	3.91	98%
8	187	6.68	84%
16	165	7.58	47%

Tabela 1: Exemplo de resultados esperados

5. **Análise de Speedup:** Gráfico demonstrando o ganho vs. ideal (Lei de Amdahl)
6. **Discussão:**
   - Por que o speedup não é linear?
   - Onde está o gargalo no seu código?
   - Como poderia melhorar ainda mais?
7. **Contribuição:** Descrição das tarefas realizadas por cada integrante

Desafio Extra: Análise de Contenção

Meça e explique a diferença de performance entre:

1. Lock dentro do loop (implementação ingênua)
2. Acumulação local + lock único no final (otimizada)

Apresente gráficos comparativos no relatório.

8. Regulamento de Apresentação e Defesa

Critérios de Avaliação

- Este trabalho deve ser feito em grupo de no mínimo 3 e no máximo 4 alunos
- **Defesa de Código:** O professor poderá solicitar:
  - Alterar o número de threads durante a execução
  - Modificar a alíquota do imposto (ex: 8%) ou outras alterações
  - Explicar o comportamento do código em diferentes cenários
  - Todos os alunos devem demonstrar domínio sobre o código
  - A apresentação é obrigatória para nota. A ausência na apresentação implica em ficar com ZERO no trabalho
- **Qualidade do Relatório:** Clareza, rigor técnico e profundidade da análise
- **Organização do Código:** Comentários, nomes de variáveis, estruturação

Atenção: Critérios Eliminatórios

- **Divergência de Valores:** Se a versão paralela apresentar um Total diferente da sequencial, o trabalho será **invalidado** (nota zero).
- É interessante calibrar o script que gera o arquivo de log para que o tempo de processamento da versão sequencial seja no mínimo igual a 5 segundos (+ ou -)
- **Plágio ou Uso de IA sem Compreensão:** O aluno deve demonstrar domínio do código durante a defesa. Cópias ou respostas "decoradas"sem entendimento resultarão em nota zero.
- **Uso de Parallel.For e outras estruturas:** A implementação deve usar `Thread` manualmente. O uso de `ThreadPool`, `Parallel.For` ou de outros recursos de paralelização ainda não estudados resultará em desconto de 80% da nota.

9. Referências Acadêmicas

[1] PACHECO, P. **Introduction to Parallel Programming**. Morgan Kaufmann, 2011.

[2] Microsoft. **Thread Class Documentation**. Disponível em: docs.microsoft.com

[3] GERGEL, V. **Introdução à Programação Paralela em C#**. Casa do Código, 2018.

9. Apêndice A: Script Gerador de Logs

Um script Python para gerar arquivos de teste está disponível no nosso ambiente. Uso recomendado:

```
python gerar_log.py 10000000 loot_log.txt
```